

features, it is important to apply feature selection techniques to prune the set of features to a smaller subset. These techniques are actually good predictors of the desired outcome variable, eliminating those attributes which are not useful in predicting the outcome – in this case, classifying whether an email is spam or not.

There are multiple ways of doing feature selection. Features can be selected as a pre-processing step independently, before selecting the learning algorithm. In this case, the preprocessing step involves analysis to determine which subset of the features are highly correlated with the desired outcome variable using standard correlation techniques such as Pearson's correlation or Linear Discriminant Analysis (LDA). Such methods are known as filter methods for feature selection since we are filtering out features which do not exhibit good correlation with the outcome variable. Another way of pruning the set of useful features is what is known as wrapper methods. In this case, we use a subset of features and train a model and measure its performance. Based on the performance results obtained, we decide whether or not to retain that subset of features in the next iteration. In simplistic terms, this can be considered as a brute force search of the feature space for finding out the optimal set of features that are useful in predicting the outcome variable.

While wrapper and filter methods explicitly perform feature selection, independent of the learning algorithm used, there are certain embedded learning methods which incorporate feature selection implicitly as part of their learning the model weights. As part of regularisation, certain learning models try to maintain the weights associated with the different features to small values, and in certain cases close or equal to zero. When the corresponding weights associated with a set of features are set to zero by the learnt model, those features are no longer used in predicting the outcome variable and hence, implicitly, these features have been eliminated though the training data still contains the values associated with them.

A good example of implicit feature selection is the Lasso regression method or the L1 norm regression method. In Lasso regression, the objective function also includes the sum of the absolute values of the weight coefficients and while minimising this term, it can set many of the weight coefficients to zero. This results in the associated features getting eliminated from being involved in the outcome variable prediction. Hence, this is an implicit form of feature selection.

While discussing various feature selection methods, related techniques to reduce the model complexity by reducing the dimensionality of the high dimensional training data include the explicit dimensionality reduction techniques such as principal components analysis. One of the typical follow-on questions that get asked when discussing dimensionality reduction and feature selection is: What is the difference between the two techniques and when would you opt for dimensionality reduction vs feature selection? The key difference between the two techniques is that feature selection does not involve any change in the feature space, whereas dimensionality reduction typically involves a change in feature space by changing the mathematical representation of the high dimensional data to a lower dimensional feature space, while maintaining the relative distance between the objects. While both

of them help to reduce model complexity and training time, feature selection is a straightforward, easy-to-interpret technique whereby we drop certain features which are not useful in predicting the outcome. This can be done using domain knowledge. On the other hand, dimensionality reduction techniques are mathematical in their approach and require mapping the original high dimensional feature space into a new lower dimensional feature space, while preserving the relative distance between the data instances during the transformation.

Another question from last month's column was about the accuracy of the classifiers. The question focused on classifiers for cancer images where the data is highly imbalanced, to the extent that accuracy may not always be the most useful criterion in selecting the optimal classifier. This is a binary classifier with the two classes being 'benign' and 'malignant'. Given that most of the radiology images in cancer tumour classification are associated with healthy human beings, the 'benign' class category is the most frequently occurring class in the data set, and there are only a few positive samples where the category is 'malignant'.

Now let us assume that we create a dummy classifier, which always predicts the outcome as 'benign'. If 99 per cent of the data set is associated with the 'benign' class, our dummy classifier will have an accuracy of 99 per cent, while it is not actually learning anything from the training data set. It is easy to see now why accuracy alone may not be a sufficient criterion for the classifier selection in this case, where the data is highly imbalanced.

There are multiple ways of dealing with this issue. One is to create a balanced data set from this imbalanced data set, and then use accuracy as the criterion in the classifier selection. Another is to use weighted class scores in evaluating the performance of the classifier wherein we give much higher weight to predicting positive classes correctly (in this example, predicting 'malignant' correctly would give greater weightage to the classifier performance, whereas predicting 'benign' correctly gives a much lower weightage, with the weightages being appropriately normalised based on the corresponding class distributions). We will discuss more about the methods and techniques for evaluating classifier performance in next month's column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyaswam_AT_yahoo_DOT_com. Till we meet again next month, wishing all our readers a wonderful and productive month ahead! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist at Conduent Labs India (formerly, Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group [Computer Science Interview Training India](http://www.linkedin.com/groups?home=&gid=2339182) at <http://www.linkedin.com/groups?home=&gid=2339182> and <http://www.linkedin.com/groups?home=&gid=2339182>.